

# Banco de Dados MySQL

Um guia introdutório: conceitos, sintaxe e tipos de dados

---

# Sumário

1. O que é um banco de dados
2. O que é o MySQL
3. Conceitos fundamentais
4. Sintaxe básica (comandos SQL)
5. Tipos de dados (tipos primitivos)
6. Exemplo prático completo

# 1. O que é um banco de dados

Um **banco de dados** é um sistema organizado para guardar, buscar e gerenciar informações de forma eficiente, muito mais rápido e confiável do que usar arquivos soltos ou planilhas. O MySQL é um **banco de dados relacional**: os dados ficam organizados em **tabelas** (parecidas com planilhas), e essas tabelas podem se relacionar entre si — por exemplo, uma tabela de "clientes" relacionada a uma tabela de "pedidos".

## 2. O que é o MySQL

O **MySQL** é um **Sistema Gerenciador de Banco de Dados** (SGBD) relacional, gratuito e de código aberto, muito usado em sites, aplicativos e sistemas empresariais. Para conversar com ele, usamos a linguagem **SQL** (Structured Query Language), com a qual criamos tabelas, inserimos dados, buscamos informações e fazemos alterações no banco.

Resumindo: **MySQL é o programa** que armazena e gerencia os dados, e **SQL é a linguagem** usada para se comunicar com ele.

## 3. Conceitos fundamentais

### Banco de dados (database)

É o "container" principal que agrupa todas as tabelas relacionadas a um mesmo sistema. Por exemplo, um banco de dados chamado Loja pode conter as tabelas de clientes, produtos e pedidos.

### Tabela (table)

É onde os dados realmente ficam guardados, organizados em linhas e colunas, assim como uma planilha.

### Coluna (column)

Representa um tipo de informação da tabela, como "nome", "idade" ou "email". Cada coluna tem um **tipo de dado** definido (veremos na seção 5).

### Linha (row / registro)

Representa um item individual guardado na tabela. Se a tabela é de clientes, cada linha é um cliente específico, com seus dados preenchidos nas colunas.

| id | nome        | email           | idade |
|----|-------------|-----------------|-------|
| 1  | Maria Silva | maria@email.com | 28    |
| 2  | João Souza  | joao@email.com  | 34    |

Exemplo de tabela "clientes": cada linha é um registro e cada coluna é um campo.

## Chave primária (Primary Key)

É uma coluna (geralmente chamada `id`) usada para identificar cada linha de forma única — não pode haver duas linhas com o mesmo valor de chave primária. É como o CPF de uma pessoa: identifica cada registro sem ambiguidade.

## Chave estrangeira (Foreign Key)

É uma coluna que faz referência à chave primária de outra tabela, criando um relacionamento entre elas. Por exemplo, a tabela "pedidos" pode ter uma coluna `cliente_id` que aponta para o `id` da tabela "clientes".

## 4. Sintaxe básica (comandos SQL)

Comandos SQL geralmente são escritos em maiúsculas por convenção (embora o MySQL não diferencie maiúsculas de minúsculas nos comandos) e terminam com ponto e vírgula ;.

### Criar um banco de dados

```
CREATE DATABASE loja;  
USE loja;
```

CREATE DATABASE cria o banco. USE seleciona qual banco será usado nos comandos seguintes.

### Criar uma tabela

```
CREATE TABLE clientes (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL,  
    idade INT,  
    data_cadastro DATE  
);
```

AUTO\_INCREMENT faz o valor de id aumentar sozinho a cada nova linha. NOT NULL obriga o preenchimento do campo.

### Inserir dados (INSERT)

```
INSERT INTO clientes (nome, email, idade, data_cadastro)  
VALUES ('Maria Silva', 'maria@email.com', 28, '2024-03-10');
```

### Consultar dados (SELECT)

```
-- Buscar todas as colunas de todas as linhas  
SELECT * FROM clientes;  
  
-- Buscar apenas nome e email  
SELECT nome, email FROM clientes;  
  
-- Buscar com uma condição  
SELECT * FROM clientes WHERE idade > 30;  
  
-- Ordenar resultados  
SELECT * FROM clientes ORDER BY nome ASC;  
  
-- Limitar quantidade de resultados  
SELECT * FROM clientes LIMIT 5;
```

### Atualizar dados (UPDATE)

```
UPDATE clientes
SET idade = 29
WHERE id = 1;
```

Sempre use WHERE em um UPDATE. Sem ele, **todas** as linhas da tabela serão alteradas!

## Excluir dados (DELETE)

```
DELETE FROM clientes
WHERE id = 1;
```

Assim como no UPDATE, esquecer o WHERE em um DELETE apaga **todas** as linhas da tabela. Cuidado redobrado aqui.

## Outros comandos úteis

| Comando       | O que faz                                                               |
|---------------|-------------------------------------------------------------------------|
| ALTER TABLE   | Modifica a estrutura de uma tabela existente (adicionar/remover coluna) |
| DROP TABLE    | Apaga uma tabela inteira (estrutura e dados)                            |
| DROP DATABASE | Apaga um banco de dados inteiro                                         |
| JOIN          | Combina dados de duas ou mais tabelas relacionadas                      |
| GROUP BY      | Agrupa linhas para cálculos (soma, média, contagem etc.)                |

## 5. Tipos de dados (tipos primitivos)

Toda coluna de uma tabela precisa de um **tipo de dado**, que define que espécie de valor ela pode guardar (número, texto, data, etc). Abaixo estão os tipos mais usados no dia a dia com MySQL.

### Números inteiros

| Tipo            | Faixa aproximada      | Uso comum                                  |
|-----------------|-----------------------|--------------------------------------------|
| <b>TINYINT</b>  | -128 a 127            | Valores pequenos, ex: idade, status (0/1)  |
| <b>SMALLINT</b> | -32.768 a 32.767      | Números pequenos a médios                  |
| <b>INT</b>      | -2,1 bi a 2,1 bi      | O mais usado para números inteiros, ex: id |
| <b>BIGINT</b>   | números muito grandes | IDs muito grandes, contadores              |

### Números decimais

| Tipo                  | Descrição                                                                                                    |
|-----------------------|--------------------------------------------------------------------------------------------------------------|
| <b>FLOAT / DOUBLE</b> | Números com casas decimais, com pequena perda de precisão. Bom para cálculos científicos.                    |
| <b>DECIMAL(p, s)</b>  | Números decimais exatos, ideais para dinheiro. Ex: DECIMAL(10,2) guarda até 10 dígitos, 2 depois da vírgula. |

### Texto (strings)

| Tipo              | Descrição                                                                              |
|-------------------|----------------------------------------------------------------------------------------|
| <b>CHAR(n)</b>    | Texto de tamanho fixo de n caracteres. Ex: CHAR(2) para sigla de estado (SP, RJ).      |
| <b>VARCHAR(n)</b> | Texto de tamanho variável, até n caracteres. O mais usado para nomes, emails, títulos. |
| <b>TEXT</b>       | Texto longo, sem limite prático. Bom para descrições, comentários, artigos.            |

### Data e hora

| Tipo             | Formato                               | Exemplo             |
|------------------|---------------------------------------|---------------------|
| <b>DATE</b>      | AAAA-MM-DD                            | 2024-03-10          |
| <b>TIME</b>      | HH:MM:SS                              | 14:30:00            |
| <b>DATETIME</b>  | AAAA-MM-DD HH:MM:SS                   | 2024-03-10 14:30:00 |
| <b>TIMESTAMP</b> | similar ao DATETIME, com fuso horário | 2024-03-10 14:30:00 |
| <b>YEAR</b>      | ano com 4 dígitos                     | 2024                |

## Outros tipos importantes

| Tipo           | Descrição                                                                                            |
|----------------|------------------------------------------------------------------------------------------------------|
| <b>BOOLEAN</b> | Verdadeiro ou falso. No MySQL é armazenado internamente como TINYINT(1) (0 = falso, 1 = verdadeiro). |
| <b>ENUM</b>    | Uma lista fixa de valores possíveis. Ex: ENUM('pendente','pago','cancelado').                        |
| <b>BLOB</b>    | Dados binários, como imagens ou arquivos.                                                            |

Dica: na dúvida entre VARCHAR e TEXT para textos curtos (nomes, títulos, emails), prefira VARCHAR com um limite razoável — é mais eficiente. Use TEXT apenas quando o conteúdo pode ser realmente longo, como um artigo ou comentário.

## 6. Exemplo prático completo

Juntando tudo o que vimos, aqui está um exemplo simples de um banco de dados de uma loja, com duas tabelas relacionadas:

```
CREATE DATABASE loja;
USE loja;

CREATE TABLE clientes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL
);

CREATE TABLE pedidos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    cliente_id INT,
    produto VARCHAR(100) NOT NULL,
    valor DECIMAL(10,2) NOT NULL,
    data_pedido DATE,
    FOREIGN KEY (cliente_id) REFERENCES clientes(id)
);

INSERT INTO clientes (nome, email)
VALUES ('Maria Silva', 'maria@email.com');

INSERT INTO pedidos (cliente_id, produto, valor, data_pedido)
VALUES (1, 'Teclado mecânico', 259.90, '2024-03-10');

-- Buscar os pedidos de cada cliente
SELECT clientes.nome, pedidos.produto, pedidos.valor
FROM pedidos
JOIN clientes ON pedidos.cliente_id = clientes.id;
```

Esse último comando usa JOIN para combinar informações das duas tabelas, mostrando o nome do cliente junto com os produtos que ele comprou.